3.1.2.1 Model mešovitog celobrojnog programiranja za optimizaciju procesa višeslojnih operacija u ranžirnim stanicama

```
from pulp import *  #uvozim PuLP biblioteku za LP
solver_path = "C:\\x64_win64\\cplex.exe" - ako koristim
eksterni solver
timeLimit = 300 #sekunde
model = LpProblem("RailYardScheduling", LpMinimize)
#Kreiram optimizacioni model sa ciljem "Minimizacija"

#====================
#ULAZNI PARAMETRI
#====================
I_max = 5 #broj dolaznih vozova
O_max = 5 #broj odlaznih vozova
B_max = 5 #broj odredišta - narednih ranžirnih stanica žel
kola, tj blokova
K_max = 5 #broj ranžirnih koloseka
T_max = 120 #"vremenski horizont - dužina - minutna
diskretizacija, 120 minuta
T = range(1, T_max+1) #diskretni vremenki trenuci, T_max +
1 jer gornja granica nije ukljucena
N_RK_max = 100 #maksimalan broj kola na jednom ranžirnom
koloseku
Tmin= 15 #vremenska razlika između prispeća kola na
ranžirni kolosek i početka njihovog formiranja
R_PRI=4 #stopa prispeća - maksimalan broj kola koji može
da uđe u sistem po jedinici vremena  #zadatak - uvesti
zavisnost od koloseka
R_OTP = 3 #stopa otpreme - maksimalan broj kola koji može
da se otpremi po jedinici vremena
N_OV_max=100 #maksimalan broj kola u sastavu odlaznog voza
N_OV_min=50 #minimalan broj kola u sastavu odlaznog voza
mi = 0.50 #tolerancija u broju kola odlaznih vozova
```

```
H_RI = 5 #minimalni interval između početaka ranžiranja
dva voza
H_TP = 15 #vreme trajanja tehničkog pregleda
H_FJ = 10 #vreme trajanja formiranja
M= 10000 #veliki broj potreban za if-then ograničenja


I = range(1, I_max+1)   # dolazni vozovi
O = range(1, O_max+1)   # odlazni vozovi
B = range(1, B_max+1)   # odredišta - broj odredišta
jednak broju koloseka u ranžirnom parku
K = range(1, K_max+1)   # koloseci
T = range(0, T_max+1)   # vreme

#Vreme prispeća vozova
T_PRI = { 1: 10, 2: 20,3: 30,4: 40, 5: 50} #minuti
#Vreme otpreme vozova
T_OTP = {1: 70, 2: 80,3: 90,4: 100, 5: 110} #minuti
#vreme trajanja ranžiranja voza i
H_RJ = {
    1: 20,
    2: 25,
    3: 15,
    4: 22,
    5: 30
}
#broj kola za blok b (odredište) dolaznog voza i -
pretpostavka je da svaki voz ima određen broj kola za
određeno odredište (u realnosti ne mora da bude slučaj)
N_IT = {
    # dolazni voz 1
    (1,1): 10, (2,1): 15, (3,1): 5,  (4,1): 8,  (5,1): 12,
    # dolazni voz 2
    (1,2): 8,  (2,2): 12, (3,2): 10, (4,2): 6,  (5,2): 14,
    # dolazni voz 3
    (1,3): 15, (2,3): 5,  (3,3): 12, (4,3): 10, (5,3): 8,
    # dolazni voz 4
    (1,4): 6,  (2,4): 14, (3,4): 9,  (4,4): 11, (5,4): 10,
```

```
    # dolazni voz 5
    (1,5): 12, (2,5): 10, (3,5): 8,  (4,5): 7,  (5,5): 13
}
#Zadatak: prilagoditi ranžirne intervale H_RI broju kola u
dolaznim vozovima
#A(k,t=0) - broj kola u početnom trenutku na svakom od
koloseka k=5
A_init = {1:5, 2:8, 3:6, 4:7, 5:4} #svaki kolosek ima neki
broj kola u početnom trenutku
#for k in K:
#    model += A[k][0] == A_init[k]

#blok-kolosek raspodela - svaki kolosek prihvata samo
jedan blok tj svaki kolosek je "rezervisan" za jednu
destinaciju
PHIkb = {
    (1,1):1, (1,2):0, (1,3):0, (1,4):0, (1,5):0,
    (2,1):0, (2,2):1, (2,3):0, (2,4):0, (2,5):0,
    (3,1):0, (3,2):0, (3,3):1, (3,4):0, (3,5):0,
    (4,1):0, (4,2):0, (4,3):0, (4,4):1, (4,5):0,
    (5,1):0, (5,2):0, (5,3):0, (5,4):0, (5,5):1
}
#Zadatak - pretpostaviti da više blokova može na jedan
kolosek ili da blok može imati više koloseka (ako ima
mnogo kola)

#matrica blok-odlazni voz, uzeće vrednost 1 kada teretna
kola sa odredištem b mogu da budu otpremljena odlaznim
vozom koji
#se otprema u vreme t - dozvoljeno je da jedan voz sadrži
jedan blok kola
sigmabt  = {
    # t = 70 (odlazni voz 1)
    (1,70):1, (2,70):0, (3,70):0, (4,70):0, (5,70):0,
    # t = 80 (odlazni voz 2)
    (1,80):0, (2,80):1, (3,80):0, (4,80):0, (5,80):0,
    # t = 90 (odlazni voz 3)
    (1,90):0, (2,90):0, (3,90):1, (4,90):0, (5,90):0,
    # t = 100 (odlazni voz 4)
    (1,100):0, (2,100):0, (3,100):0, (4,100):1, (5,100):0,
    # t = 110 (odlazni voz 5)
```

```
    (1,110):0, (2,110):0, (3,110):0, (4,110):0, (5,110):1
}
#Zadatak: Proširiti rečnik tako da jedan voz može da ima
više blokova kola
# --------------------
# VARIJABLE
# --------------------
tau_s = LpVariable.dicts("tau_s", I, lowBound=1,
upBound=T_max) #vreme početka ranžiranja dolaznog voza i
tau_e = LpVariable.dicts("tau_e", I, lowBound=1,
upBound=T_max) #vreme završetka ranžiranja dolaznog voza i
sii = LpVariable.dicts("s", (I, I), 0, 1, cat="Binary")
#binarna promenljiva niza ranžirnih zadataka
#biće 1 ako i ide ispred j (pri čemu je i prispeo pre j,
oba pripadaju skupu dolaznih vozova I), 0 u suprotnom
#Zadatak: kako izgleda matrica s za pretpostavljeno
rešenje sekvenciranja
xit = LpVariable.dicts("x", (I, T), 0, 1, cat="Binary")
#binarna indikator promenljiva koja ukazuje na završetak
aktivnosti
#ranžiranja - ranžiranje voza i je završeno u vreme t
Akt = LpVariable.dicts("Akt", (K, T), lowBound=0)
#kumulativni broj kola koja su ranžirana na kolosek k
#u svakom trenutku t, t>0
At = LpVariable.dicts("At", T, lowBound=0)
Dkt = LpVariable.dicts("D", (K, T), lowBound=0)
#kumulativni broj kola koja se "izvlače" sa ranžirnog
koloseka k
Dt = LpVariable.dicts("Dt", T, lowBound=0)
ykt = LpVariable.dicts("y", (K, T), 0, 1, cat="Binary")
#binarna promenljiva koja pokazuje status zauzetosti
ranžirnog koloseka k aktivnostima
#na formiranju odlaznog voza
fu = LpVariable.dicts("fu", (B, T), lowBound=0) #broj kola
sa odredištem b koja se u vreme t prerađuju lokomotivom za
formiranje u ranžirnom parku
#NAPOMENA: blok je direktno vezan za kolosek, nema potrebe
imati k kao indeks!
fiz = LpVariable.dicts("fiz", (B, T), lowBound=0)  #broj
kola sa odredištem b koja se premeštaju lokomotivom za
formiranje ka otpremnom parku
```

```
T_dep = list(T_OTP.values()) #skup vremena odlaznih vozova
(T_OTP = {1: 70, 2: 80,3: 90,4: 100, 5: 110})
diz = LpVariable.dicts("diz", (B, T_dep), lowBound=0)
#broj kola sa odredištem b spremnim za odlazni voz u vreme
otpreme odlaznog voza
#odlazni voz je implicitno definisan preko vremena T_dep
nod = LpVariable.dicts("nod", O,  lowBound = 0,
cat="Integer")#broj kola koji je formiran u odlazni voz o
#Broj kola u vozu = suma kola po blokovima u tom trenutku
"""for o in O:
    t = T_OTP[o]
    model += nod[o] == lpSum(diz[b][t] for b in B)"""
#Broj kola sa odredištem b koja se nalaze u otpremnom
parku u vreme t
stbt = LpVariable.dicts("stbt", (B, T), lowBound=0,
cat="Integer")
print(list(diz.keys())[:10])
# --------------------
# FUNKCIJA CILJA
# --------------------
#ukupni dolazni tokovi/kumulativni do vremena t
for t in T:
    model += At[t] == lpSum(
        N_IT[b, i] #broj kola sa dolaznog voza i za blok b
        for i in I if T_PRI[i] <= t
        for b in B
    )

#ukupni odlazni tokovi/kumulativni do vremena t
for t in T:
    model += Dt[t] == lpSum(
        diz[b][tprim]
        for b in B
        for tprim in T_dep if tprim <= t #mora T_dep jer
je diz definisano u odnosu
        #na T_dep
    )

model += lpSum(
    At[t] - Dt[t]
    for t in T
```

$$Min\ W=\sum_{t=0}^{T}[A(t)-D(t)]$$

```
)


#===========
#OGRANIČENJA
#==========

#1 Ograničenja zadataka na ranžiranju
#1.1 Ograničenja tehničkog pregleda
for i in I:
    model += tau_s[i] >= T_PRI[i] + H_TP
    #1.2 Ograničenja trajanja ranžiranja #PROBLEMATIČNO
    # for i in I:
    #     model += tau_e[i] == tau_s[i] + H_RJ[i]
#1.3 Ograničenja intervala sleđenja i sekvenciranja pri
obavljanju uzastopnih
#zadataka na ranžiranju
for i in I:
    for j in I:
        if i < j:
            model += tau_s[j] >= tau_e[i] + H_RI - M*(1 -
sii[i][j])
            model += tau_s[i] >= tau_e[j] + H_RI -
M*(sii[i][j])
#1.4 Ograničenja očuvanja toka povezana sa ranžiranjem
#stanje u trenutku t na koloseku k jednako je stanju u
prethodnom trenutku na istom koloseku
#uvećanom za broj kola koja su pristigla u trenutku t
for k in K:
    for t in T[1:]:
        model += Akt[k][t] >= Akt[k][t-1] + \
            lpSum(
                xit[i][t] *
                lpSum(N_IT.get((b,i),0) * PHIkb[(k,b)] for
b in B) #po vozu i koliko kola ide na kolosek k
                for i in I #sabiranje preko vozova - broj
kola iz voza i na koloseku k
            )
# #Primer
# """voz i = 1 ima:
# 10 kola za blok 1
```

$$\tau^s(i) \geq T^{PRI}(i) + H^{TP},\ \forall i$$

$$\tau^s(j) - \tau^e(i) \geq H^{RI} - [1 - s(i,j)] \times M,\ \forall i, j,\ i < j$$

$$A(k,t) - A(k,t-1) = \sum_{i=1}^{I} \sum_{b=1}^{B} \left[x(i,t) \times N^{IT}(b,i) \times \varphi(k,b) \right],\ \forall k,\ t \geq 1$$

```
# 5 kola za blok 2
# PHI:
# blok 1 → kolosek 1
# blok 2 → kolosek 2
# Za kolosek 1 10 * 1 + 5 * 0 = 10
# Za kolosek 2: 10 * 0 + 5 * 1 = 5
# """
# #po modelu iz knjige: lpSum(xit[i][t] *
N_IT.get((b,i),0) * PHIkb[(k,b)]
#       # for i in I for b in B) - ispravna takođe, gornja
je usvojena zbog čitljivosti
# #N_IT.get((b,i),0) - get komanda uzima vrednost za kljuc
(b,i) iz rečnika N_IT, ako ne postoji vraća 0
# #PHIkb.get((k,b), 0), Sigmabt.get((b,t), 0) ovako
eliminišemo runtime greške - nema nedostajucih key-value
parova
# #Alternativna formulacija ovog ograničenja:
# """total = 0
# for i in I:
#     inner_sum = sum(N_IT.get((b,i),0) * PHIkb[(k,b)] for
b in B) #koliko kola iz voza i ide na kolosek k
#     total += xit[i][t] * inner_sum"""  #da li se to
dešava u t

#1.5 Ograničenja povezanosti indikatora završenog stanja
xit i vremena završetka ranžirnog zadatka tau_e
for i in I:
    model += tau_e[i] == lpSum(xit[i][t]*t for t in T)
for i in I:
    model +=  lpSum(xit[i][t] for t in T) <= 1 #stavio sam
da voz mora da bude ranžiran umesto <=, vraćam na <=
#postavljam da je xit jednako 0 pre nego voz i stigne i
odradi tehnički pregled
#ovde bi moglo obuhvatiti i vreme trajanja ranžiranja
for i in I:
    for t in T:
        if t < T_PRI[i]+H_TP:
           model += xit[i][t] == 0
#1.6 Prostorna ograničenja kapaciteta ranžirnog koloseka
for k in K:
    for t in T:
```

$$A(k,\ t) - D(k,\ t) \le N_{\max}^{RK},\ \forall k,\ t$$

```
        model += Akt[k][t] - Dkt[k][t] <= N_RK_max
#1.7 Ograničenja aktivnosti na formiranju vozova
for t in T:
    if t + H_FJ - 1 <= T_max:
        model += lpSum(ykt[k][tprim] for k in K for tprim
in range(t,t+H_FJ)) <= 1
#range(t, t + H_FJ) → interval dužine H_FJ, sabiram sve
aktivnosti na ovom intervalu
#Zadatak: preformulisati ograničenje da važi za svaki
kolosek posebno

#1.8 Ograničenja stope odlaznih tokova u ranžirnoj stanici
for k in K:
    for t in T:
        if t >= 1:
           model+= Dkt[k][t]-Dkt[k][t-1] <=
N_RK_max*ykt[k][t]

for k in K:
    for t in T:
        if t >= 1:
           model+= Dkt[k][t]-Dkt[k][t-1] <= Akt[k][t-1]-
Dkt[k][t-1]

#1.9 ograničenja očuvanja toka između "sortiranja" (koje
nije obuhvaćeno modelom) i formiranja:
#važi ako je jedan kolosek jedan blok
for b in B:
    for t in T:
        if t>=1:
            model += fu[b][t] == lpSum((Dkt[k][t]-
Dkt[k][t-1])*PHIkb.get((k,b), 0) for k in K)

#1.10 Ograničenja vremena trajanja zadatka na formiranju -
odnos između vremena početka i završetka formiranja
for b in B:
    for t in T[H_FJ:]:
        model += fiz[b][t] <= fu[b][t - H_FJ]

# 1.11 veza između formiranja i otpreme
```

```
# po svakom bloku (indirektno koloseku) broj kola koji
ulazi u otpremni park u t
# plus broj kola  sa odredištem b koji je već u otpremnom
parku jednak je broju kola koji je u otpremnom parku
# plus broj kola sa odredištem b koji se otprema u
trenutku t
for b in B:
    for t in T_dep: #zbog diz
        model += fiz[b][t]+stbt[b][t-1] ==
stbt[b][t]+diz[b][t]

for b in B:
    for t in T[H_FJ:]:
        model += fiz[b][t] <= N_RK_max * lpSum(ykt[k][t-
H_FJ]*PHIkb.get((k,b), 0) for k in K)

# #1.12 Ograničenje alociranja bloka na odlazni voz
for b in B:
    for t in T_dep:
        model += diz[b][t] <= N_OV_max * sigmabt[(b,t)]

# 1.13 Ograničenja broja kola odlaznog voza
for o in O:
    t=T_OTP[o] #vreme otpreme odlaznog voza
    model += nod[o] == lpSum(diz[b][t] for b in B)

#1.14 maksimalan i minimalan broj kola u odlaznom vozu
for o in O:
    model += nod[o] <= N_OV_max

for o in O:
    model += nod[o] >= N_OV_max*mi
#ako bih koristio not za promenljivu, to bi bila greška
jer je not rezervisana reč

# REŠAVANJE

model.writeLP("modelMultiLevelClass.txt")

if os.path.exists(solver_path):
  if "cplex" in solver_path:
```

```
    model.solve(apis.CPLEX_CMD(path=solver_path,
timeLimit=timeLimit))
    print("Solving with: cplex")
  elif "scip" in solver_path:
    model.solve(apis.SCIP_CMD(path=solver_path,
timeLimit=timeLimit))
    print("Solving with: scip")
else:
  model.solve(pulp.PULP_CBC_CMD(timeLimit=timeLimit))
  print("Solving with: cbc")

for v in model.variables():
    if v.varValue is not None and abs(v.varValue) > 1e-6:
        print(v.name, "=", v.varValue)

print("=== MODEL INFO ===")
print("Broj promenljivih:", len(model.variables()))
print("Broj ograničenja:", len(model.constraints))
```

Ili

```
# REŠAVANJE

model.writeLP("modelMultiLevelClass.txt")

model.solve(apis.SCIP_CMD(timeLimit=timeLimit))

for v in model.variables():
    if v.varValue is not None and abs(v.varValue) > 1e-6:
        print(v.name, "=", v.varValue)

print("=== MODEL INFO ===")
print("Broj promenljivih:", len(model.variables()))
print("Broj ograničenja:", len(model.constraints))
```